F1.5 — coordination comments, ready to paste

Four comments to post before writing any F1.5 server code. Each one asks a specific person for a specific decision, with a proposed default so nobody has to invent an answer from scratch. Post 1 and 4 on #16 itself; post 2 and 3 on the teammate's issue so it lands in their notifications.

Once all four have a reply, this thread is the coordination evidence for the sprint review.

Comment 1 — on #13 (F1.2, schema) · for @Brian-Two

> **F1.5 needs four things in the `users` schema — cheaper to add now than to migrate later**
>
> Hi Brian — I own #16 (server-side auth), which writes into the `users` table your ERD defines. Four asks, all small, and all much cheaper to land in your review than as a follow-up migration:
>
> 1. **`users.password_hash TEXT NOT NULL`** — and please do *not* carry over a `password` column. Today `AppUser.password` (`src/data/appDatabase.ts:9`) is stored in plaintext; the column name change is a useful tripwire against anyone reintroducing it.
> 2. **`users.role`** — constrained to `student` and `org_officer`, defaulting to `student`. Per ADR 0002 the server assigns this and never accepts it from the client.
> 3. **`CREATE UNIQUE INDEX ON users (lower(email))`** — the current code normalizes email case in application logic (`appDatabase.ts:162`, `:183`). That belongs in the database, or two accounts differing only in case will eventually both exist.
> 4. **A `sessions` table** — `id` (primary key, random), `user_id` (FK, cascade on delete), `created_at`, `expires_at`, `revoked_at NULL`, plus an index on `user_id`. ADR 0002 chose database-backed sessions over JWTs specifically so that the logout and revocation acceptance criteria on #16 are satisfiable.
>
> Also worth noting for your own criteria: there is no `password_hash` anywhere in the repo today, so nothing needs migrating. Per ADR 0002 Decision 5 we are dropping the two seed accounts rather than rehashing them.
>
> Full reasoning in `docs/adr/0002-authentication.md`. Anything here that conflicts with your ERD, tell me and I will adapt rather than fork.

Comment 2 — on #14 (F1.3, API service and Postgres) · for @oanyamele

> **Three questions I need answered before I can start F1.5's server code**

Hi Ogechi — #16 (server-side auth) has to live inside the process this issue creates, so I am blocked on you for three decisions. I have proposed a default for each; if you just reply "all three as proposed" I can start.

1. **Framework and language.** Whatever you pick, I follow. My only request is that it be TypeScript so I can share zod schemas between server and client — `zod@3.25` is already a dependency, so validation logic can be written once instead of twice.
2. **Where auth code lives.** Proposing `server/src/routes/auth.ts`, `server/src/lib/password.ts`, `server/src/lib/session.ts`, and `server/src/middleware/requireAuth.ts`. If your layout differs, give me the paths and I will match it — I would rather conform than have two conventions in one server.
3. **The cookie question, which is the one that can actually bite us.** ADR 0002 chose an httpOnly session cookie. If the API and the SPA are served from the **same site** on staging, `SameSite=Lax` works and there is nothing to do. If they are on **different origins**, the cookie must be `SameSite=None; Secure`, the API must send `Access-Control-Allow-Credentials: true` with an explicit origin allowlist, and the client must send `credentials: "include"` on every request. That is a real change to your CORS setup, so I would rather know now than discover it during the demo. Which is it?

One more thing I need from you: **who owns the session-signing secret and the database URL as environment variables**, and where they are stored for staging. I will not commit secrets, so I need to know the mechanism before Phase 2.

Schedule note. If this issue has not landed by the end of Week 3, I am going to build the four auth endpoints against an in-memory store behind F1.4's repository interface, with the real hashing, real cookies, and real rate limiting, and swap the store when you land. That keeps F1.5 off your critical path and the swap touches one file. Flagging it so it does not look like duplicated work when you see the PR.

Comment 3 — on #15 (F1.4, repository interface) · for @neaylasimone

The three `UserRepository` methods F1.5 needs

Hi Neayla — I own #16, and the client half of it depends on the `UserRepository` from this issue. Right now `src/pages/Login.tsx:9` imports `authenticateUser` and `createUser` straight out of `src/data/appDatabase`, which is exactly the coupling you are removing. Here is the surface I need so you only have to shape it once:

```
interface UserRepository {
  signup(input: { email: string; password: string; name: string }): Promise<SafeUser>;
  login(input: { email: string; password: string }): Promise<SafeUser>;
  logout(): Promise<void>;
  me(): Promise<SafeUser | null>;
}
```

Three notes on the details, which matter more than the names:

- **`SafeUser` is `{ id, email, name, role, createdAt }`.** No `password`, no `password_hash`, not ever. Please export exactly one serializer that produces it, so the shape cannot drift and accidentally start including a credential.

- `signup` **takes no** `role`**.** Per ADR 0002 Decision 4, the server always creates a `student`; the client is not allowed to ask for `org_officer`. This closes a privilege-escalation hole in the current sign-up screen (`src/pages/Login.tsx:66`, `:80`, `:29`) where the user picks their own role.
- `me()` **has no** `HttpRepository` **shortcut.** It must be a real request. The whole point of F1.5 is that the client is no longer the authority on who is logged in, so this cannot be answered from cache or local storage.

For your `LocalStorageRepository` (the offline and test adapter): it should store the `SafeUser` only and should **fail** on `login`, not fake one. An offline adapter that authenticates is the defect we are removing.

Happy to write these four methods against your interface myself once it is merged — tell me which you prefer.

Comment 4 — on #16 (F1.5) · for @oanyamele, tagging F19

The session claim contract F19 will authorize against

@oanyamele — you own F19 (role-based access control), which needs a trustworthy identity on the request. F1.5 produces it. Posting the contract here so you can build against it instead of inventing one and reconciling later.

After `requireAuth` runs, the request carries:

```
interface SessionClaim {
  userId: string;
  email: string;
  role: "student" | "org_officer";
  sessionId: string;
  expiresAt: string;   // ISO 8601, UTC
}
```

Four properties of it that matter for your work:

1. `role` **is read from the database on every request**, not from the cookie or a token. So if an officer is demoted, the next request reflects it immediately. This is the main reason ADR 0002 chose database-backed sessions over JWTs.
2. **F1.5 ships** `requireAuth` **and** `requireRole(...roles)` as middleware. `requireAuth` is authentication only — is there a valid session. Everything about *what this user may do*, including resource ownership, is yours. I am deliberately not building an ownership check, because a half-built one in two places is worse than one built well in yours.
3. **Unauthenticated is** `401`**, authenticated-but-forbidden is** `403`**.** Let us agree on this now so the client can distinguish "sign in again" from "you cannot do that."
4. **Audit logging.** F1.5 logs signup, login success, login failure, logout, and role change with actor, action, timestamp, and IP. If you want authorization *denials* in the same table, tell me the shape and I will make the writer reusable rather than have you build a second one.

> One thing I want to be explicit about, because it affects how you read my PR: after F1.5, `ProtectedRoutes` in `src/App.tsx` is **user experience only**. It exists so users are not shown a screen that will fail. It is not a security boundary and must never be treated as one — the boundary is your server-side checks. I will say the same thing in the PR description and in a comment on the component.